

人工智能公开课

1. 开场破冰 + 认知对齐

(1) 自我介绍 (2 分钟)

“大家晚上好，我是Paul老师，之前做过多个大模型微调项目，开发过智能体，做过数学建模及数据分析与预测、做过机器视觉识别项目，帮企业落地过 RAG 知识库，今天带大家绕开我踩过的坑，让大家更快速的了解人工智能开发”

“如果你会用 Python 写 for 循环、判断、调用 requests 库，今天这节课能让你带走一个能聊天的本地大模型；如果不会，也能看懂 AI 开发的逻辑，后续补完 Python 基础再看我们这次回放自己试一下。”

(2) AI基本介绍+学员互动 (6 分钟)

区分AI应用和AI开发的区别

其实AI应用很早就有了，像我们的人脸识别、语言翻译之类的都是比较早就有，只是在2022年大模型的应用热度起来之后AI又有一个大的提升。

大家经常说学AI、学AI，那我们先要分清楚一下学AI，是学什么？

一个是AI应用，AI应用就是常见用AI对话或工具来AI问答、AI作图、AI制作视频、AI写文章等，我们统称为AIGC；

一个是AI开发，AI开发就是利用AI技术开发出这些应用方向，或者是基于某些方向的垂直应用开发；那么AI开发有哪些方向呢？我们我这里按技术方向来列一下：

可以打开这些模型社区 看tasks(任务) <https://huggingface.co/models> <https://www.modelscope.cn/models>

机器视觉：其实就是基于图形图像的识别与处理；例如：人脸识别、扫码数烤串的签、车牌识别、医学影像分析、目标跟踪、基于视觉的自动驾驶、图片处理、老照片修复等，像比较流行的YOLO模型；

自然语言处理（大模型应用）：让机器“理解和生成”人类语言，其实就是我们常用的大模型 Deepseek、豆包、chatgpt等，能对话、推理、编程、内容生成等，其实我们平时用Deepseek、豆包的那些网页APP可以算是智能体应用，而他们底层使用的才是大语言模型；

视频与音频处理：让机器“听懂”和“说出”声音。语音识别：将语音转换为文本，用于语音助手、会议纪要。语音合成：生成高度自然、富有情感的人声，用于有声书、虚拟主播。音频事件检测：识别环境声音（如玻璃破碎声、枪声）用于安防。音乐生成与分析：AI作曲、音乐风格分类、歌曲推荐。

决策与推荐系统：让机器“做出最优决策”或“推荐你喜欢的”。推荐系统：电商平台（淘宝）、内容平台（抖音、Netflix）的商品和内容推荐。欺诈检测：金融领域识别信用卡盗刷、保险欺诈。风险控制：信贷审批中的信用评分。游戏AI：AlphaGo、AlphaStar，在复杂环境中做出超越人类的决策。

机器人技术与自动化（具身智能）：将AI与物理世界连接，工业机器人、服务机器人（送餐机器人、导览机器人、家庭清洁机器人、扫地机器人）、自主系统：（无人机、自动驾驶汽车、自动导引车）

强化学习：让机器通过“试错”自我学习。训练AI玩电子游戏（如Dota 2, StarCraft II）。机器人控制让机器人学习复杂的动作技能，如行走、抓取。资源管理（数据中心冷却节能、电网负荷分配）。

抛出问题，让学员在公屏扣答案，快速筛选需求：

1. “用过AIGC就是用AI问答做图、做视频之类扣 1，了解过AI开发的扣 2”。
2. “想做大模型应用的扣 A，数据分析挖掘的扣 B，机器视觉、图像处理扣C，想做具身智能的扣D，都想要的扣 E”；

(3) AI 学习痛点拆解 + 课程价值预告+ (10分钟)

痛点共鸣：

大家在学习AI开发的过程中，有遇到什么问题？可以在公屏中打出来

(这时候水军可以发出一些提问)

水：不会部署

水：被一堆专业名词吓退了

水：感觉要数学很牛的才能做

水：有时候看到很多资料，看到很多技术名称，但是不知道他们互相有什么联系

水：之前试着自己部署一些项目，一直部署不起来

水：想做一些项目实战，但是这查一下资料、那查一下资料，然后就偏离轨道，走弯路，还没搞下去

我们总结一些刚刚的问题，大部分想学 AI的同学都卡在这几个地方：

- 1、本地部署大模型，半天装不好环境；
- 2、看了一堆的理论，不知道怎么用在实际问题中；
- 3、听了一堆的技术名称、不知道什么意思；
- 4、想做项目，不知道从‘基础算法’到‘大模型 Agent’的路径怎么走。

价值预告：

“今天我们先解决前3个问题 ——技术名称解析并梳理他们之间的关系、跑通本地大模型聊天机器人，看几个 AI 场景的代码演示；最后告诉你第4个问题的答案，也就是怎么从‘入门’到‘能做项目’的学习路径”。

AI技术讲解：

刚刚有同学说AI的一堆名称，像 **LLMs**（大模型）、**Multimodality**（多模态）、**Agent**（智能体）、**MCP**（模型上下文协议）、**Prompt**（提示词）、**Workflow**（工作流）、**Tool calling**（工具调用）、**Context**（上下文）都不太清楚分别是什么意思，对应什么关联；

那我们先以Agent为入口来把这些串起来；

Agent，其实就是基于AI的智能应用体的一种，例如我们用的Deepseek、豆包的在线聊天或APP，都可以认为是智能体；更复杂的好智能体就是会自动根据配置的节点**Workflow（工作流）**进行自动化处理我们要处理的事情，如果用过coze、dify、n8n的话，可能会更好理解；

其实学术界对智能体有着更高的定义，就是用户输入需求，智能体会自动匹配需求去调用模型、工具完成一些事情，然后验证结果是否正确，不正确又会自己调整，再调用，而不是根据工作流去执行。

一般**智能体**包括（**模型、记忆、工具**）这三个基础模块

模型就是我们的**LLM（大语言模型）**支持 文本输入，文本输出，后面发展到**Multimodality LLMs（多模态大模型）**支持（文本、图片、音频、视频）--> 文本输出，后面就 图片生成模型（文生图、图生图），视频生成模型（文生视频、图生视频等）

Memory（记忆） 大模型是没有记忆的，有同学可能会说，我之前用了那么多AI聊天，都能记住之前我问的，那是这些平台已经做好了记忆的构造部分，然后把我们的聊天记录统一发给了大模型，这样大模型就有了记忆，

我们每次问模型的时候就是 **Prompt 提示词**，为了更好通过AI获取到要的内容，这时候就会需要规范一下提示词，例如口吻、身份、条件之类的，很多智能体在这一块做的都很好

如果我们把所有的聊天记录都传给 大模型，我们聊的越多，就要传越多，这样可能不行的，所以后面就出来了 **context（上下文）工程**，会把聊天记录中重要的信息提炼出来给到大模型

随着大模型的能力越来越强，这时候希望大模型可以有更多的手和脚，例如 Deepseek里的联网查询，这种就是典型的**Tool calling（工具调用）**，还有他里面的文档读取工具，这就是为什么Deepseek不是多模态大模型，也可以支持文档和图片；

随着工具越来越多，很多公司都在做很多的工具，像PPT设计、邮件工具等等，为了方便大模型调用这些工具，就出了一个**MCP（Model context Protocol）模型上下文协议**，规定了模型的输入和输出的内容格式，有人把这个比喻成USB，有了这个协议后，所有模型调用工具就变得简单方便了，其实智能体也可以变成一个MCP

2. 实战 1：本地大模型聊天机器人（30 分钟，核心环节）

好了，我们上面大概了解了AI基础知识，因为时间关系，我们后面还有干货，就不能给大家详细讲下去了，接下来我们边实操边嵌入一些知识点；

我想大家都有用过 Deepseek、豆包之类的，那些是不是都是线上直接用的？Deepseek还偶尔会卡，有些敏感行业还不想把自己的数据传上去，所以就有了本地部署的需求，我们今天就简单的实现一下 Deepseek、豆包这样的本地AI聊天系统。

在做本地部署前，我们先来了解一下这三个LM开源的平台 huggingface hub 全球最大的LM开源发布平台，各种模型都有 <https://huggingface.co/>

modelscope 阿里出品，也是各种模型都有 <https://www.modelscope.cn/>

Ollama Meta出品，目前是放的大语言模型 <https://www.ollama.com/>

部署工具：

Huggingface Transformers

vLLM

Ollama

...

(1) Ollama 部署 (10 分钟)

步骤拆解:

1. 安装 Ollama:

官网下载: <https://ollama.com/download>

因为要从github上下载, 所以最好走VPN, (Edeg插件ilink) <https://github.com/ollama/ollama/releases>

默认是安装到C盘的, 如果不想占用C盘空间, 可以用下面的方法安装到其他盘

在其他盘新建一个目录, 例如 D:/ollama

打开命令行 OllamaSetup.exe /DIR=D:\ollama

2. 拉取轻量级模型:

(解释一些参数含义、蒸馏的意思)

B, 代表十亿 (Billion), 蒸馏可以相当于是基于某个模型的再学习, 降低参数量达到原来模型的差不太多的效果

32B为320亿参数, 70B为700亿参数

(选7B以下模型, 避免电脑配置不够, 解释“0.5B 参数, 4G 内存就能跑, 适合入门”)

终端输入ollama pull qwen3:0.6b;

3. 测试模型:

终端输入ollama run qwen3:0.6b, 输入“你好”, 看模型回复 (如果卡了, 排查“终端没启动服务”“网络没拉取模型”, 现场演示问题解决)。

关键提醒: “如果拉取模型慢, 课后发国内镜像源; 如果电脑内存小, 用llama2:7b-chat-q4_0 (4 位量化, 更省内存)”。

(2) Python 调用 Ollama API (15 分钟)

- 第一步: 写基础调用脚本 (边写边解释核心代码):

```
import requests # 用于发API请求
import json

def chat_with_ollama(prompt):
    url = "http://localhost:11434/api/chat" # Ollama默认API地址
    headers = {"Content-Type": "application/json"}
    data = {
        "model": "qwen3:0.6b", # 刚才拉取的模型名
        "messages": [{"role": "user", "content": prompt}],
        "stream": False # 关闭流式输出, 适合入门
    }
    response = requests.post(url, headers=headers, data=json.dumps(data))
    return response.json()["message"]["content"]
```

```
# 测试
if __name__ == "__main__":
    user_input = input("你: ")
    reply = chat_with_ollama(user_input)
    print(f"AI: {reply}")
```

- 解释重点：“Ollama 的 API 是 RESTful 风格，只需要传模型名、用户 prompt，就能拿到回复，比调用远程 API 更灵活（不用等 key，不用怕限流）”。
- 现场运行：输入“用 Python 写一个快速排序”，看 AI 回复代码，然后运行 AI 给的代码，验证“AI 输出的代码能跑通”（强化“实用”属性）。

- 第二步：加简单 GUI 界面（5 分钟，提升成就感）：

用tkinter加一个输入框 + 按钮 + 输出框，代码片段：

```
import tkinter as tk
from tkinter import scrolledtext

def send_msg():
    user_input = entry.get()
    if not user_input:
        return
    output_box.insert(tk.END, f"你: {user_input}\n")
    reply = chat_with_ollama(user_input) # 复用上面的函数
    output_box.insert(tk.END, f"AI: {reply}\n\n")
    entry.delete(0, tk.END)

# 界面搭建
root = tk.Tk()
root.title("本地AI聊天机器人")
entry = tk.Entry(root, width=50)
entry.pack(pady=10)
send_btn = tk.Button(root, text="发送", command=send_msg)
send_btn.pack()
output_box = scrolledtext.ScrolledText(root, width=60, height=20)
output_box.pack(pady=10)

root.mainloop()
```

- 演示效果：输入“解释什么是线性回归”，AI 给出通俗回复，让学员直观看到“自己写的程序能和 AI 聊天”。

(3) 学员问题答疑（5 分钟）

- 收集公屏问题：比如“Ollama 启动报错怎么办？”“模型回复慢怎么优化？”“能换其他模型吗？”，现场解答 + 记录，课后整理成“常见问题手册”（作为报名福利）。

3. 核心技术场景拆解（30 分钟，选 3 个高价值方向）

每个场景“1 分钟讲原理 + 5 分钟看代码 + 4 分钟聊落地”，不深钻理论，只展示“技术能解决什么问题”

(1) 场景 1：Python 做数据预测（线性回归，10 分钟）

- 原理（1 分钟）：“线性回归就是找X 和 Y 的线性关系’，比如‘房屋面积（X）→房价（Y）’，用 Python 的 sklearn 库能 10 行代码出结果”。
- 代码演示（5 分钟）：

```
from sklearn.linear_model import LinearRegression
import numpy as np

# 数据：房屋面积（m²）→房价（万元）
X = np.array([50, 70, 90, 110]).reshape(-1, 1) # 特征必须是2D数组
y = np.array([150, 210, 270, 330])

# 训练模型
model = LinearRegression()
model.fit(X, y)

# 预测：130m²的房价
predict_X = np.array([130]).reshape(-1, 1)
predict_y = model.predict(predict_X)
print(f"130m²房价预测: {predict_y[0]:.2f}万元")
```

- 演示效果：输出“390.00 万元”，解释“模型学到了‘每增加 10m²，房价涨 60 万’的规律，实际项目中会用更多特征（如楼层、地段）”。
- 落地案例（4 分钟）：“某房产中介用这个逻辑做‘房价初筛’，用户输入面积、地段，先给一个预测价，再推真实房源；当然这个带大家了解一些简单数据分析，实际应用中还有怎么处理异常数据（比如老破小单价高）？怎么评估模型准不准的问题。

(2) 场景 2：NLP 文本分类（用迁移学习，10 分钟）

- 原理（1 分钟）：“文本分类就是‘给文本贴标签’，比如‘邮件→垃圾 / 非垃圾’‘评论→正面 / 负面’，用迁移学习（比如 BERT 模型）不用从头训练，10 分钟能跑通”。
- 代码演示（5 分钟）：

用transformers库的pipeline（极简接口）：

conda安装：

```
conda install -c pytorch pytorch
conda install -c huggingface transformers
```

```
from transformers import pipeline
import os

# 设置使用国内镜像
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'

# 加载预训练模型（情感分析：正面/负面）
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

# 测试文本
texts = ["This AI course is amazing!", "I hate fixing bugs in code."]
```

```
results = classifier(texts)
for text, res in zip(texts, results):
    print(f"文本: {text} → 情感: {res['label']}, 置信度: {res['score']:.4f}")
```

pip安装:

```
pip install torch transformers huggingface_hub
```

pip安装后的强制指定镜像

```
import os
# 设置使用国内镜像
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'

import huggingface_hub
from transformers import pipeline

# 强制设置镜像
huggingface_hub.constants.HF_HUB_URL = 'https://hf-mirror.com'
```

当然你也可以不用动态导入模型，直接预下载好后，本地导入模型

- 演示效果：
输出“amazing→POSITIVE (0.9998) ”“hate→NEGATIVE (0.9994) ”，
解释“distilbert 是轻量级 BERT，比原版小 60%，适合入门”。
- 落地案例（4 分钟）：
“某电商用这个做‘评论分析’，自动统计‘差评关键词’（比如‘物流慢’），每周给运营出报告；我们可以使用中文模型（如 bert-base-chinese）做商品评论分类”“怎么用自己的数据微调模型”。

场景 3：多模态图像生成（Stable Diffusion，10 分钟）

- 原理（1 分钟）：
“Stable Diffusion 是‘文本生成图像’的模型，输入‘a cat wearing a hat’，就能出图，本地用diffusers库能调用”。
Diffusers 是一个开源的深度学习库，专门用于处理 **扩散模型（Diffusion Models）**。
- 代码演示（5 分钟）：

```
import os
# 在导入任何其他库之前设置
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'

from diffusers import StableDiffusionPipeline
import torch

# 设置设备
```

```

# 有GPU用cuda, 没有用cpu (慢一点)
if torch.cuda.is_available():
    device = "cuda"
    torch_dtype = torch.float16 # 用16位精度, 省显存
elif torch.backends.mps.is_available():
    device = "mps"
    torch_dtype = torch.float16 # 用16位精度, 省显存
else:
    device = "cpu"
    torch_dtype = torch.float32

print(f"使用设备: {device}")
# 加载模型 (选轻量级的, 如CompVis/stable-diffusion-v1-4)
pipe = StableDiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4",
    torch_dtype=torch_dtype,
    use_safetensors=True
).to(device)

# 文本生成图像
prompt = "A cute dog playing with a ball in a park, sunny day, cartoon style"
image = pipe(prompt, num_inference_steps=20).images[0]

# 保存图像
image.save("dog_play.jpg")
print("图像已保存为dog_play.jpg")

```

- 演示效果：打开生成的dog_play.jpg，展示“文本描述→图像”的效果，解释“num_inference_steps 越小越快，越大越清晰”。
- 落地案例（4 分钟）：“某设计工作室用这个做‘初稿生成’，客户说‘要一个科技感的 logo’，先让 AI 出 10 张图，再人工修改，效率提升 50%；现在因为这节课我们没太多时间，如果要深入控制的话还有这些场景‘怎么控制图像风格（如动漫 / 写实）’‘怎么用图生图（上传一张照片，让 AI 改成卡通）’”。

4. 课程介绍 + 学习路径（10 分钟）

好了，我们今天主要先给大家展示了一些常用的应用场景，因为AI的内容比较多，我们今天这样一节课是没办法让大家学的很深入的，像数据处理那里我们还有很多算法的，不只是线性回归，我们怎么深度挖掘数据还需要掌握更多的知识点；例如Agent开发、微调、RAG之类的应用，我们需要更多的时间来讲了。如果大家的人工智能深入开发感兴趣的，平时有遇到一些困惑的地方，可以在公屏问一下

(简单互动)

水：机器学习什么时候讲？

水：深度学习和机器学习有什么关系？

水：怎么学习Agent开发吗？

水：微调和RAG的差别

答：微调像是给模型“回炉重造”，教它新的知识或技能，改变模型本身。RAG 像是给模型配一个“外部知识库”，在需要时去查阅，不改变模型本身。

水：数学重要吗？非学不可吗？

答：人工智能这行发展相当迅速，在实际工作中肯定要边干边学，至少有些数学公式你要能认识，如果要往越深入发展，数学用到就越多，你看看那些人工智能的论文就可以发现了

水：如何学数学？要定一个长期计划吗？

答：不需要从头开始花大量时间一步一个脚印去学，用到什么查什么，不要去解题，我们不是去考试，目前先理解一个公式是做什么的，有什么用 就足够了；

水：有哪些数学知识是必须的？

答：

知识点	内容	作用
高等数学	高等数学基础，微积分，泰勒公式与拉格朗日，	机器学习公式推导必备
线性代数	线性代数基础，特征值与矩阵分解，	算法求解必备
概率论	概率论基础，随机变量与概率估计，常用分布	机器学习经常提这些词
统计分析	回归分析，假设检验，相关分析，方差分析	数据分析必备

水：有了深度学习还需要机器学习吗？

答：深度学习可以说是机器学习算法的一种，并不是有了神经网络其他经典算法就不需要了，需要根据不同的任务和数据来选择最合适的算法，学习路径肯定是先从机器学习开始，其实掌握了这些经典算法之后再看神经网络真的很简单！

(引出 人工智能实战营)

看大家对人工智能的深入学习有比较大的兴趣，刚好我们要开一个人工智能实战营的课，我顺便讲一下我们的课程大纲，你们也可以通过学习大纲了解一下 AI开发的学习路径，注意不要一开始就去啃论文，先熟悉应用场景，边熟悉，边查资料。

(拉出海报开始介绍)

课后配套准备 (提升转化)

今天讲的代码笔记，我课后整理好，可联系班主任发送“AI资料”可领；

(引导留资，后续跟进)

继续互动答疑，引导报名

水：上课时间是怎么样的？

答：我们一周三节课，目前是安排在周135 晚上7:30 到9:30

水：实战需要掌握哪些技能？

答：在实战中可能把数学知识点都弱化了，因为更多时候我们都是使用现成的工具包来完成任务（调包侠）。这里需要大家掌握的节能功能比较多，首先就是熟练使用这些常用工具包了，数据预处理，特征工程，调参，验证这些都是非常核心的步骤。概括来说就是要完成不同的任务所需流程和套路都是类似的，但是使用的方法和算法却可能不同，这就需要大家不断积累来丰富实战经验了。

水：这些案例能让我收获什么？

答：熟悉常见的AI开发场景，并积累实战经验，为后续发展做好准备，也可以把学到技术应用到实际的生产环境中

水：AI的后续的发展是怎么样的？不可能大家都去做大模型吧

答：**模型轻量化与边缘化**：不可能所有场景都依赖超大规模模型，未来会出现更多轻量级模型，部署在本地或边缘设备上。既降低了成本，又增强了隐私保护。

多模态融合：未来的 AI 不仅能理解文字，还能同时处理图片、音频、视频、传感器数据，实现真正的“全感知”。

自主学习与进化：AI 将不仅仅依赖预训练，还会具备持续学习、在线学习能力，更好地适应动态环境。

垂直领域的应用、智能体开发、AGI

水：深度学习都需要学什么呢？

答：深度学习可以说是当下最好用的算法了，各个领域都能吃得开。其实最核心的还是在计算机视觉和自然语言处理中，因为神经网络算法更适用于图像和文本数据。主要需要掌握的就是算法和框架了，算法就是CNN,RNN这些经典网络模型，框架就是实战的工具了例如tenorflow,Pytorch等，后面还会详细说。

水：深度学习听起来比较高大上，是不是比机器学习难很多？

答：好像现在好多小伙伴一拿到任务，第一个想法都是直接用深度学习做。如果深度学习难度大，做起来麻烦，那还能有这么高的热度吗？其实恰恰相反，我觉得深度学习 真的比机器学习简单很多，在机器学习中需要我们对不同的数据选择不同的预处理方法和特征工程构建方法。深度学习中的套路相对来说更固定一些，而且有这些开源框架和各大经典网络架构，我们通常需要做的就是套用就可以了。整体难度要比机器学习任务更容易一些（只是相对来说！）。

水：这么多项目，有没有哪个是需要重点学习的？最好能写在简历里面呢？

答：目前AI Agent的应用市场比较大，当然计算机视觉一直都比较多岗位，这些项目都可以写到简历上